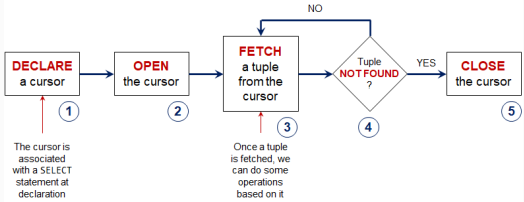


C.2. Stored Procedures

A **stored procedure** is a function that does not return any value.

```
CREATE OR REPLACE PROCEDURE <pname>
( <param> <type>, <param> <type>, ... )
AS $$
DECLARE
  <var> <type>;
  :
BEGIN
  <code>
END;
$$ LANGUAGE <language>;
```

C.3. Cursors



```
-- Find the largest increase in price between versions
CREATE OR REPLACE FUNCTION max_increase(gname VARCHAR(32))
RETURNS NUMERIC AS $$
DECLARE
  cur CURSOR (vname VARCHAR(32)) FOR
    SELECT g.price
    FROM games g
    WHERE g.name = vname
    ORDER BY g.version ASC;
  res NUMERIC;
  prev NUMERIC;
  curr NUMERIC;
BEGIN
  OPEN cur(gname);
  res := 0; prev := 0;
  LOOP
    FETCH cur INTO curr;
    EXIT WHEN NOT FOUND;
    IF (curr - prev) >= res
    THEN res := (curr - prev);
    END IF;
    prev := curr;
  END LOOP;
  CLOSE cur;
  RETURN res;
END; $$ LANGUAGE plpgsql;
```

Directions:

- **NEXT/FORWARD**: To the next row (default)
- **PRIOR/BACKWARD**: To the previous row
- **FIRST**: To the first row
- **LAST**: To the last row
- **ABSOLUTE n**: To the index n (starting from 0, can be negative)
- **RELATIVE n**: Forward by n rows (or backward if negative)

C.4. Triggers

Trigger function specifies the action that is executed when the event occurs (what to do). It returns a TRIGGER.

- Checks for violations using an SQL query.
- If violation is detected, raise exception to roll back the operation. Otherwise, return NEW to allow the operation to proceed.

Trigger determines when it should fire by binding a trigger function to operations on a table.

```
CREATE TRIGGER <name>
<timing> <event> ON <table>
FOR EACH <granularity>
[WHEN <condition>]
EXECUTE FUNCTION <fname>();
```

Event: INSERT, UPDATE, DELETE (can be chained with OR)

Transition variables:

- **NEW**: the modified row after the triggering event
- **OLD**: the modified row before the triggering event

Timing:

- **BEFORE**: trigger executes **before** operation is done, can validate/modify the tuple before operation takes place
Note: For row-level BEFORE triggers, returning NULL signals database to skip the operation for that specific row.

- **AFTER**: trigger executes **after** operation is done

Return value:

- For BEFORE triggers, returning NEW allows the operation to proceed (possibly with modifications to the tuple being inserted/updated); returning NULL cancels the operation for that tuple.
- For AFTER triggers, the return value has no effect.

	NULL returned	Non-NULL returned
BEFORE INSERT	No tuple inserted	Tuple inserted
BEFORE UPDATE	No update	Tuple updated
BEFORE DELETE	No deletion	Deletion proceeds
AFTER trigger	No effect	No effect

Granularity:

- **FOR EACH ROW**: row-level, executes once for each affected tuple (has access to NEW and OLD)
- **FOR EACH STATEMENT**: executes once per SQL statement, regardless of how many rows are affected (no access to NEW and OLD)

Order of activation:

1. BEFORE statement-level trigger
2. BEFORE row-level trigger
3. AFTER row-level trigger
4. AFTER statement-level trigger

Within each category, triggers are executed in **alphabetical order** of name.

Trigger functions can access:

- TG_OP: type of operation that activated the trigger
- TG_TABLE_NAME: table name that activated the trigger

Example:

```
-- Trigger function
CREATE OR REPLACE FUNCTION fr21()
RETURNS TRIGGER AS $$
BEGIN
  IF EXISTS (
    SELECT c.customerid
    FROM customers c NATURAL JOIN downloads d
    WHERE d.name = 'Domainer'
    AND EXTRACT(year FROM AGE(c.dob)) < 21
  ) THEN
    RAISE EXCEPTION 'Underaged!'; -- STOP!
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
-- Trigger
CREATE CONSTRAINT TRIGGER tr21
AFTER INSERT OR UPDATE ON downloads
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE PROCEDURE fr21();
-- (deferrable: checked only at the end of the transaction)
```

D. Functional Dependencies and Normal Form

D.1. Functional Dependencies

- $X \rightarrow Y$ is a **functional dependency** if for every tuples t_1, t_2 :

$$\text{If } t_1.X = t_2.X, \text{ then } t_1.Y = t_2.Y.$$

- **Closure** A^+ of attribute A is set of all attributes implied by A
- **Armstrong's axioms**:
 - Axiom of reflexivity: If $X \supseteq Y$, then $X \rightarrow Y$.
 - Axiom of augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z .
 - Axiom of transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
- Rule of decomposition: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- Rule of union: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- By definition, to prove that $X \rightarrow Y$ holds, we need to show that $Y \in X^+$.
- **Algorithm: Compute closure**
 1. Initialise the closure to $\{A_1, A_2, \dots, A_n\}$.
 2. If there is a FD $A_i, A_j, \dots, A_m \rightarrow B$ such that $A_i, A_j, \dots, A_m$ are all in the closure, then put B into the closure.
 3. Repeat Step 2, until we cannot find any new attribute to put into the closure.
- X is a **superkey** if X implies all attributes
- A **key** is a minimal superkey (A superkey is a superset of a key)
- **Algorithm: Finding keys**
 1. Compute closure of each attribute subset.
 2. Identify superkeys based on the closures.
 3. Identify keys from the superkeys.

Tip 1: attribute not on RHS must be in the key
Tip 2: check small attribute sets first
- **Prime attribute**: attribute that appears in a key

D.2. BCNF

- An FD $X \rightarrow A$ is **decomposed** if A is a single attribute. An FD $X \rightarrow A$ is **trivial** if $A \in X$.
- How to find non-trivial and decomposed FDs:
 1. Compute closure of each attribute subset.
 2. From each closure, remove trivial attributes.
 3. Derive non-trivial and decomposed FDs from each closure.
- R is in **BCNF** if for every non-trivial and decomposed FD $X \rightarrow A$, X is a superkey.
- **Algorithm: BCNF check**
 1. Compute closure of each attribute subset.
 2. Check if there is a "more but not all" closure.
 3. If such a closure exists, then R is not in BCNF.
- **Algorithm: Decomposition algorithm**

Suppose R is not in BCNF.

 1. Find a subset X of attributes of R , such that X^+ contains more attributes than X , but not all attributes in R .
 2. Decompose R into two tables R_1 and R_2 , such that
 - R_1 contains all attributes in X^+ ,
 - R_2 contains all attributes in X as well as attributes not in X^+ .
 3. If R_1 is not in BCNF, further decompose R_1 . If R_2 is not in BCNF, further decompose R_2 . (To check this, to derive the closures on a table R_i decomposed from a table R , compute closure of each attribute subset, then restrict to R_i)

Note: R with exactly two attributes is in BCNF.

Check if a decomposition is **lossless**:

Common attributes in R_1 and R_2 constitute a superkey of R_1 or R_2

Check if a decomposition is **dependency-preserving**:

1. Compute closure of each attribute subset X using Σ , restrict to R_i
2. Derive projected FDs Σ_{R_i}
3. Decomposition is dependency-preserving if union of all Σ_{R_i} is Σ

D.3. 3NF

- R is in **3NF** if for every non-trivial and decomposed FD $X \rightarrow A$, (1) either X is a superkey, or (2) A is a prime attribute.
- 3NF is more "lenient" than BCNF.
- **Algorithm: 3NF check**
 1. Compute closure for each attribute subset, then find the keys.
 2. For each FD, check if LHS is a superkey, or each attribute on RHS is prime attribute.
 3. If all FDs satisfy this condition, then R is in 3NF.
- Given a set F of FDs, the **minimal basis** M of F is a simplified version of F satisfying the following conditions:
 1. M is equivalent to F .
 2. Every FD in M is a non-trivial and decomposed FD.
 3. No FD in M is redundant (i.e., cannot be derived from other FDs in M).
 4. For each FD in M , none of the attributes on left-hand side is redundant.

That is, if we remove an attribute from left-hand side, then the resulting FD is a new FD that cannot be derived from the original set of FDs.
- **Algorithm: Derive minimal basis**

Given a set F of FDs.

 1. Simplify RHS: Decompose FDs so that each right-hand side contains only one attribute.
 2. Simplify LHS: Remove redundant attributes on left-hand side of each FD.
 3. Remove redundant FDs.
- A **canonical cover** is obtained from a minimal basis by combining FDs whose left-hand sides are the same.
- **Algorithm: Synthesis algorithm**

Suppose R is not in 3NF.

 1. Find a minimal basis of F .
 2. Calculate the canonical cover.
 3. For each FD, create a table that contains all attributes in the FD.
 4. Check if *any* of the tables contain a key for R ; if not, then create a table that contains a key for R .
 5. Remove subsumed tables.